Topic 1: Analysis of Algorithms

#eda #apunts #english #PARCIAL1

1. Computation Time & Efficiency

Goals of Algorithmic Analysis

- **Compare algorithmic solutions** to determine which approach is superior for a given problem.
- **Predict resource usage** (time and memory/space) prior to implementation.
- **Improve existing algorithms** by identifying performance bottlenecks.

Key Observations on Efficiency

- Efficiency depends inherently on the **input size** n5.
- Execution time is relative to external factors (hardware, compiler, operating system), but these factors only exert a **linear impact** on overall performance.
- **Input Size** $|x|$: The number of symbols needed to encode an input x.
  - *Unary encoding for natural numbers*: $|15| = 15$.
  - *Binary encoding for natural numbers*: $|15| = 4$ (since $15_{10} = 1111_2$).
  - *Lists / Vectors*: The number of components n.

Cases of Analysis

- **Worst-case cost** $T_{\text{worst}}(n)$: $T_{\text{worst}}(n) = \max\{T(x) \mid x \in \mathcal{A} \wedge |x| = n\}$. Establishes an upper bound on resources that the algorithm will **never exceed**.
- **Best-case cost** $T_{\text{best}}(n)$: $T_{\text{best}}(n) = \min\{T(x) \mid x \in \mathcal{A} \wedge |x| = n\}$. Represents the minimum cost, but is generally of little practical utility.
- **Average-case cost** $T_{\text{avg}}(n)$: $T_{\text{avg}}(n) = \sum_{x \in \mathcal{A}, |x|=n} \Pr(x) T(x)$, where $\Pr(x)$ is the probability distribution over all inputs of size n. Often difficult to compute precisely.

---

2. Asymptotic Notation & Growth Rates

Asymptotic notation classifies functions according to their **relative growth rates** for sufficiently large inputs, ignoring constant coefficients.

Definitions

1. **Big-$\mathcal{O}$ Notation (Asymptotic Upper Bound)**:

$$\mathcal{O}(g) = \{f : \mathbb{N} \to \mathbb{R}^+ \mid \exists c \in \mathbb{R}^+, \exists n_0 \in \mathbb{N}, \forall n \geq n_0 : f(n) \leq c \cdot g(n)\}$$

2. **Big-Ω Notation (Asymptotic Lower Bound)**:

$$\Omega(g) = \{f : \mathbb{N} \to \mathbb{R}^+ \mid \exists c \in \mathbb{R}^+, \exists n_0 \in \mathbb{N}, \forall n \geq n_0 : f(n) \geq c \cdot g(n)\}$$

3. **Big-Θ Notation (Asymptotically Tight Bound)**:

$$\Theta(g) = \{f : \mathbb{N} \to \mathbb{R}^+ \mid \exists c_1, c_2 \in \mathbb{R}^+, \exists n_0 \in \mathbb{N}, \forall n \geq n_0 : c_1 g(n) \leq f(n) \leq c_2 g(n)\}$$

Key Properties & Limit Rules

- **Fundamental Relation**: $f \in \Theta(g) \iff f \in \mathcal{O}(g) \text{and} f \in \Omega(g)$.
- **Duality**: $f \in \Omega(g) \iff g \in \mathcal{O}(f)$.
- **Limit Criteria**:

$$\lim_{n \to \infty} \frac{f(n)}{g(n)} = \begin{cases} 0 \implies f \in \mathcal{O}(g) \text{but} f \notin \Omega(g) \\ \infty \implies f \in \Omega(g) \text{but} f \notin \mathcal{O}(g) \\ c \in (0, \infty) \implies f \in \Theta(g) \end{cases}$$

- **Sum Rule**: $\mathcal{O}(f) + \mathcal{O}(g) = \mathcal{O}(\max(f, g))$.
- **Product Rule**: $\mathcal{O}(f) \cdot \mathcal{O}(g) = \mathcal{O}(f \cdot g)$.
- **Polynomials**: For $p(n) = a_k n^k + \cdots + a_0$ with $a_k > 0$, $p(n) \in \Theta(n^k)$.
- **Logarithmic Change of Base**: $\log_a(n) \in \Theta(\log_b(n))$ for $a, b > 1$.
- **Asymptotic Hierarchies**:
  - Logarithmic vs. Polynomial: $(\ln n)^a \in \mathcal{O}(n^b)$ for $a, b > 0$.
  - Polynomial vs. Exponential: $n^b \in \mathcal{O}(c^n)$ for $c > 1$.

Hierarchy of Common Complexity Classes

1. **Constant** $\Theta(1)$: Parity check.
2. **Logarithmic** $\Theta(\log n)$: Binary search.
3. **Linear** $\Theta(n)$: Sequential search in an unsorted vector.
4. **Quasilinear** $\Theta(n \log n)$: Optimal comparison-based sorting.
5. **Quadratic** $\Theta(n^2)$: Sum of square matrices, Selection Sort.
6. **Cubic** $\Theta(n^3)$: Standard matrix multiplication.
7. **Polynomial** $\Theta(n^k)$: Combination enumeration.
8. **Exponential** $\Theta(k^n)$: Exhaustive search in configuration spaces.

---

3. Cost Analysis of Algorithms

Analysis of Iterative Algorithms

- **Basic Operations** $\Theta(1)$: Primitive assignments, arithmetic operations, variable increments/decrements, reference parameter passing, and array indexing.
- **Vector Creation/Copying**: Copying or returning a vector of size n takes $\Theta(n)$ time.
- **Sequential Statements (~~F1 F2~~)**: Additive cost $C_1 + C_2$.
- **Conditionals (`if (B) F1; else F2`)**: Bounded by ~~$+$~~ $\max(C_1, C_2)$, where ~~$+$~~ is the evaluation cost of condition B.
- **Loops (`while (B) F`)**: Total cost is ~~$\left(\sum_{k=1}^{N}(C_k + $~~ ~~$k$~~ ~~$)\right) + $~~ ~~$N+1$~~, summing iteration execution costs and condition re-evaluations.
- **Examples**:
  - **Selection Sort**: $T(n) = \Theta(1) + \sum_{i=1}^{n}(\Theta(i) + \Theta(1)) = \Theta\left(\frac{n(n+1)}{2}\right) = \Theta(n^2)$.
  - **Insertion Sort**: Worst-case is $\Theta(n^2)$ (reverse sorted input), best-case is $\Theta(n)$ (already sorted input).

Analysis of Recursive Algorithms & Recurrences

Recursive costs are expressed using **recurrence relations** $T(n)$.

- **Recursive Linear Search**: $T(n) = T(n-1) + \Theta(1) \implies T(n) \in \Theta(n)$.
- **Recursive Binary Search**: $T(n) = T(n/2) + \Theta(1) \implies T(n) \in \Theta(\log n)$.

---

4. Master Theorems for Recurrences

Master Theorem I: Subtraction Recurrences

Applies to recurrences of the form:

$$T(n) = a \cdot T(n-c) + g(n) \quad \text{for} g(n) \in \Theta(n^k), c \geq 1$$

.

$$T(n) \in \begin{cases} \Theta(n^k), & \text{if} a < 1 \\ \Theta(n^{k+1}), & \text{if} a = 1 \\ \Theta(a^{\frac{n}{c}} n^k), & \text{if} a > 1 \end{cases}$$

- *Example*: $T(n) = T(n-1) + \Theta(n) \implies a = 1, c = 1, k = 1 \implies T(n) \in \Theta(n^2)$.

Master Theorem II: Division Recurrences

Applies to divide-and-conquer recurrences of the form:

$$T(n) = a \cdot T(n/b) + g(n) \quad \text{for} g(n) \in \Theta(n^k), b > 1$$

Let $c = \log_b(a)$:

$$T(n) \in \begin{cases} \Theta(n^k), & \text{if } c < k \\ \Theta(n^k \log n), & \text{if } c = k \\ \Theta(n^c), & \text{if } c > k \end{cases}$$

- *Example (Merge Sort)*: $T(n) = 2T(n/2) + \Theta(n) \implies a = 2, b = 2, k = 1 \implies c = \log_2(2) = 1 \implies T(n) \in \Theta(n \log n)$.

---

5. Key Theoretical Results & Applications

Case Study: Computing Fibonacci Numbers

$(f(k) = f(k-1) + f(k-2))$

1. **Naïve Recursive**: $T(k) = T(k-1) + T(k-2) + \Theta(1) \implies T(k) \in \Theta(\varphi^k)$, where $\varphi = \frac{1+\sqrt{5}}{2} \approx 1.618$ (exponential cost).
2. **Iterative Dynamic Approach**: Computes values sequentially in a single loop $\implies T(k) \in \Theta(k)$.
3. **Quick Exponentiation via Matrix Multiplication**:

$$\begin{pmatrix} f(k+1) \\ f(k) \end{pmatrix} = \begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix}^k \begin{pmatrix} f(1) \\ f(0) \end{pmatrix} \implies \begin{pmatrix} f(n+1) & f(n) \\ f(n) & f(n-1) \end{pmatrix}^k$$

   Using recursive power squaring ($T(k) = T(k/2) + \Theta(1)$), computation takes $\Theta(\log k)$ **time**.

Lower Bound for Comparison-Based Sorting

- **Theorem**: Any sorting algorithm based strictly on key comparisons has a worst-case time complexity of $\Omega(n \log n)$.
- **Proof Sketch (Decision Trees)**:
  - Any execution of a comparison sort corresponds to a path in a binary decision tree where each leaf represents one of the $n!$ possible permutations of n elements.
  - A binary decision tree of height d has at most 2^d leaves.
  - Thus, $2^d \geq n! \implies d \geq \log_2(n!)$.
  - Using Stirling's approximation / logarithmic bounds:

$$\log_2(n!) \geq \log_2\left(\left(\frac{n}{2}\right)^{n/2}\right) = \frac{n}{2}\log_2\left(\frac{n}{2}\right) \in \Omega(n \log n)$$